

Concurrent Multi-Platform Social Media Extraction and LLM Sentiment Analysis: A Museum Design Case Study

Justin Lucero, Jhonatan Tacuri, and Wilmer Merchán

Universidad Politécnica Salesiana, Cuenca, Ecuador
jlucero1@est.ups.edu.ec, jtacuric@est.ups.edu.ec,
wmerchang@est.ups.edu.ec

Abstract. Social media is a primary channel through which citizens react to public decisions, but analyzing that reaction at scale is hard: opinions are scattered across platforms, arrive as unstructured text, and need context-aware interpretation rather than keyword counting. This paper presents a web application that extracts, classifies, and interprets social media opinions concurrently, applying parallel computing and natural language processing end to end. Given a search query, the system launches one thread per source and scrapes Facebook and TikTok (Selenium over a persisted Chrome profile) together with YouTube and Reddit (official REST APIs), coordinating the platforms that require manual login through a shared lock so other threads keep working. The consolidated corpus is then classified by sentiment with a large language model (OpenAI GPT-4o-mini or Groq Llama-3.1-8b-instant) through a producer-consumer thread pool, and a rule-based storytelling module turns the statistics into a narrative interpretation, served through an interactive dashboard. We evaluate the system on a real case study: public reaction to “Ecos del Sol”, the winning design for Ecuador’s new National Museum. On a corpus of 508 opinions from three concurrently running sources, classification processed the full corpus in 49.59 s and revealed a predominantly negative sentiment (54.7%) concentrated on YouTube. A controlled benchmark on an 18-record dataset shows parallel classification reaches a $2.80\times$ speedup with OpenAI – near the theoretical maximum for three threads – while Groq’s free tier yields $0.25\times$ (a slowdown) under concurrent rate limiting, a limitation often overlooked in I/O-bound parallel designs. Cross-checking the corpus with both providers yields 83.3% inter-model agreement, used as a reliability proxy absent human-labeled ground truth. These findings show that thread-based concurrency is effective for I/O-bound social media pipelines, but its benefit is conditioned by the downstream API’s rate-limiting policy, a factor that belongs in the architectural decision, not as an afterthought.

Keywords: Parallel computing · Social media mining · Sentiment analysis · Large language models · Concurrency · Explainable AI

1 Introduction

Social platforms such as Facebook, TikTok, YouTube, and Reddit have become one of the largest sources of publicly available opinion data, capturing how citizens react to political, cultural, and infrastructure decisions in near real time. This makes them attractive for public-opinion research, but also hard to analyze: the volume and velocity of new posts outpace manual review, the text is short, informal, and multilingual, and each platform has its own access mechanism, data schema, and anti-automation defenses. A system that wants to answer a question such as “*what do people think about X*” therefore has to solve three problems at once – collecting data from several heterogeneous sources fast enough to be useful, turning unstructured text into a structured sentiment signal, and presenting the result in a way a non-technical reader can interpret – without any of the three becoming the bottleneck.

This work uses as its case study the public reaction to “*Ecos del Sol*”, the winning design of an international architecture competition for Ecuador’s new National Museum, which received substantial criticism and support across social media after being announced. The research question we address is: *how can a parallel-computing-based system extract, process, and analyze large volumes of social media data, classifying sentiment and producing interpretable results efficiently, while applying core computer science and software engineering principles to the design?*

We propose and implement a web application that answers a user-provided search query end to end. On submission, the backend launches a thread per social platform (Facebook, TikTok, YouTube, and Reddit) so that extraction overlaps rather than executes sequentially; Facebook and TikTok are scraped through Selenium over a real, persisted Chrome profile, while YouTube and Reddit are queried through their official REST APIs. The consolidated, de-duplicated corpus is then classified by sentiment with a large language model (LLM) using a second, independent thread pool built around a producer–consumer queue, one worker per source. A lightweight, rule-based storytelling module turns the resulting aggregate statistics into a qualitative narrative (dominant sentiment, cross-platform contrast, engagement-weighted signal, and textual evidence), and all of this is exposed through an interactive dashboard that lets a user explore individual opinions together with the LLM’s justification for each label.

The main contributions of this paper are: (1) a concurrent, multi-source social media extraction pipeline that unifies browser-automation and official-API sources under a single thread-based architecture with shared login coordination; (2) a parallel, fault-tolerant sentiment classification stage using a producer–consumer pattern, benchmarked against its sequential counterpart; (3) an automatic storytelling component that converts aggregate sentiment statistics into a human-readable interpretation; and (4) an empirical evaluation on a real 508-opinion corpus that quantifies both the benefit and a concrete limitation of thread-based parallelism when the downstream LLM API enforces rate limits.

The rest of this paper is organized as follows. Section 2 reviews recent related work on LLM-based sentiment analysis, explainability, and parallel/distributed

processing of social media data. Section 3 describes the system architecture, the concurrency strategy used at each stage, and the technical decisions behind them. Section 4 reports the classification accuracy proxy, the case-study results, and the execution-time measurements, including the speedup benchmark. Section 5 summarizes the main findings and outlines recommendations for future work.

2 Related Work

Table 1 summarizes six recent works (2023–2025) relevant to the two axes of this paper – LLM-based sentiment analysis and parallel processing of social/textual data – and positions our system with respect to each of them.

Beyond these six, the literature confirms two long-standing points our design builds on. First, lexicon-based sentiment tools such as VADER [8] and AFINN [11] are fast but struggle with sarcasm, negation, and domain-specific jargon common in social media comments about public infrastructure – one of the reasons we adopt an LLM-based classifier instead. Second, the transformer architecture [17] and the pretraining paradigms it enabled, from BERT [3] to instruction-tuned model families such as Llama 3 [5] and GPT-4o [12], are what make it practical to classify sentiment with a short natural-language prompt (a technique surveyed by Sahoo et al. [14]) rather than a hand-labeled training set. Regarding data collection, Brown et al. [1] discuss the ethical and legal considerations of web scraping for research, which informed our decision to scrape only public search results through a user-authenticated session rather than bypassing platform authentication. Finally, Yang and Liu [18] review how social media data has been used in urban design and landscape research more broadly, providing additional context for our architecture-related case study, and Hjelle et al. [7] provide experimental evidence that dashboard visualizations measurably improve analytical decision making, which motivated the design of our exploration and storytelling dashboard.

3 Methodology

3.1 System Architecture

Figure 1 shows the end-to-end architecture. The user submits a search query and a set of sources through the web front end; a FastAPI backend creates a background job and immediately returns a job identifier, so the HTTP request never blocks on the (potentially minutes-long) extraction and classification work. The job runs in its own thread and drives three sequential stages, each of which is internally parallel: concurrent extraction, parallel sentiment classification, and storytelling generation. Progress is exposed through a polling endpoint that the dashboard consults every two seconds, including a login checkpoint that the two Selenium-based sources may need the first time they run.

Table 1. Recent related work (2023–2025) and its relation to this paper.

Work	Problem / Method	Relation to our system
Kim and Kim [9]	Measures how closely GPT-3.5/GPT-4 Turbo reproduce human-expert sentiment labels on 1,000 Facebook/Twitter posts (few-shot prompting).	Validates our core design choice of using an LLM (GPT-4o-mini, Llama-3.1-8b-instant) instead of a human-coded gold set; motivates our inter-model agreement analysis (Section 4) as a reliability proxy.
Sun et al. [16]	Proposes a two-LLM negotiation (generator/discriminator) framework to reduce single-pass sentiment errors on ambiguous text.	An alternative to our single-pass classification step; a natural extension for future work when higher reliability outweighs latency and cost.
Herrera-Poyatos et al. [6]	Surveys the “model variability problem” in LLM-based sentiment analysis and argues explainability is necessary for trustworthy deployment.	Justifies the explainability feature of our dashboard, which shows the LLM’s textual justification next to every label.
Chen et al. [2]	Combines aspect-based sentiment analysis with a SHAP-interpretable classifier on social media reviews of cultural heritage sites.	Closest domain match (public sentiment toward a heritage/architecture project); differs in using a single-platform corpus and classical ML with post-hoc explainability instead of multi-platform LLM classification.
Li and Wang [10]	Mines social media reviews to characterize citizen perception of urban public spaces in Shanghai.	Single-city, single-corpus analogue of our approach, generalized here to four concurrently polled platforms and a national landmark.
Dixit et al. [4]	Uses Python multiprocessing to run classifier hyperparameter sweeps concurrently, reducing training time.	A CPU-bound parallelism precedent; complementary to our I/O-bound design, which parallelizes network-bound LLM API calls with threads rather than processes (Section 3).
Pehlivan et al. [13]	Describes a national-scale, multi-platform media observatory with per-platform crawlers and a schema-normalization layer.	The closest architectural analogue at a much larger scale; our system targets a lighter, single-topic deployment with a producer–consumer sentiment stage and storytelling layer that the observatory does not include.

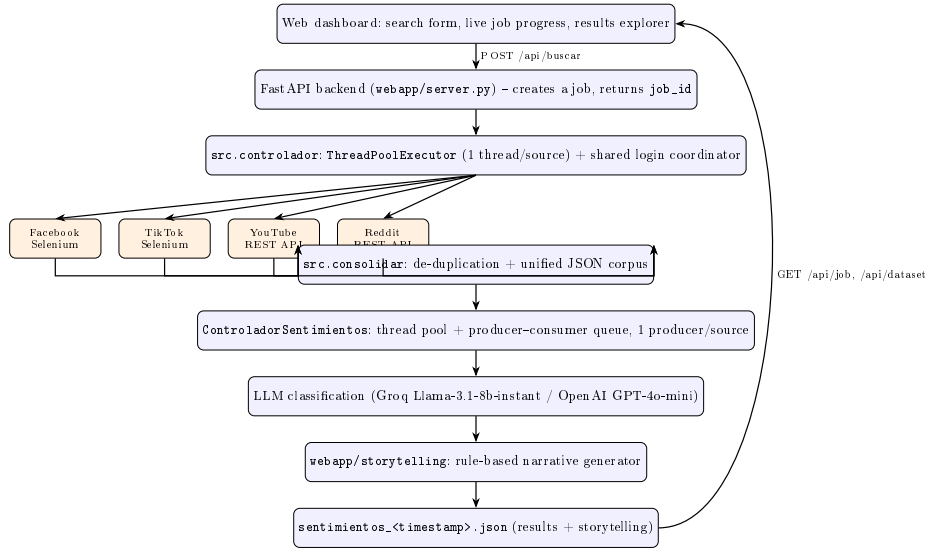


Fig. 1. End-to-end architecture: concurrent extraction (four threads, one per source), parallel sentiment classification (producer–consumer thread pool), storytelling generation, and the dashboard that consumes the result.

3.2 Concurrent Data Extraction

The extraction stage runs one thread per social platform inside a **ThreadPoolExecutor**. Facebook and TikTok block traffic from third-party scraping libraries, so both are automated with Selenium driving a real Google Chrome instance over a *copy* of the user’s Chrome profile (cookies and session data), which makes the automated browser appear as an already-known user and substantially reduces anti-bot challenges. A per-platform copy is required (rather than the real profile directly) because Chrome locks a profile directory while it is in use, and both platforms run concurrently. YouTube and Reddit, by contrast, expose official REST APIs (YouTube Data API v3, and Reddit’s OAuth `client_credentials` application-only flow) that require no browser and no user session, so those two threads never block on user interaction.

Because Facebook and TikTok may require a one-time manual login, a shared **GestorLogin** object coordinates it with a lock and an event: only one thread logs in at a time, and every other thread blocks at a checkpoint until that login finishes, avoiding two simultaneous prompts competing for the user’s attention. In the web application, this checkpoint is exposed as a job state (`login_pendiente`) that the dashboard surfaces as a button; clicking it releases a `threading.Event` that the waiting scraper thread is blocked on, so the CLI’s `input()`-based flow and the web flow share the same coordination primitive with no code duplication (Section 4 was collected with the CLI variant; the web variant follows the identical thread-per-source design).

We use **threads, not processes**, because the workload is I/O-bound: each scraper spends most of its time waiting for the browser or the network, not computing. Python releases the Global Interpreter Lock (GIL) during I/O system calls [15], so while one thread is waiting, another can make progress; wall-clock time therefore tends toward the slowest single source rather than the sum of all sources. Threads also share memory, letting all four scrapers use the same `GestorLogin` instance without inter-process serialization overhead, which a **multiprocessing**-based design would require.

Each source writes its own JSON file; `src.consolidar` then merges all four into one corpus, flattening every post and comment into a traceable record (`fuelle`, `consulta`, `texto`, `autor`, `metricas`, `id_unico`) and removing duplicates by content hash.

3.3 Parallel Sentiment Classification

The classification stage (Section 4 reports its measured speedup) follows the same I/O-bound argument, implemented as a **producer–consumer** pattern. The corpus is grouped into one block per source; a `ThreadPoolExecutor` launches one producer thread per block, and each producer calls the LLM (Groq Llama-3.1-8b-instant or OpenAI GPT-4o-mini, selected by the user) once per text, pushing the resulting `RegistroSentimiento` onto a shared, thread-safe `queue.Queue`. A dedicated consumer thread drains the queue until it has received one completion signal per producer. Sources with a large volume (more than 15 records, e.g., hundreds of YouTube comments) are further split across a nested sub-pool of up to 12 workers, so a single overloaded source cannot stall the others. Every LLM call requests a strict JSON response (`response_format={"type": "json_object"}`) with one of five categories – *positivo*, *negativo*, *neutral*, *mixto*, or *no_clasificable* – plus a short justification string, and falls back to *no_clasificable* on any parsing or network failure instead of aborting the batch, so one bad response cannot bring down the whole classification run.

3.4 Storytelling and Explainability

Two complementary explainability mechanisms close the loop between raw classification output and an interpretable result. At the record level, every label carries the LLM’s own natural-language justification, shown next to the original text in the dashboard, addressing the model-variability and trust concerns raised by Herrera-Poyatos et al. [6]. At the corpus level, a deterministic, rule-based storytelling module (`webapp/storytelling.py`) computes the dominant sentiment and its share, contrasts the dominant sentiment across sources to highlight where opinion diverges most, re-ranks the corpus by engagement (likes) to distinguish an isolated opinion from one that resonates with the audience, and quotes one or two real LLM justifications as textual evidence – all without an additional LLM call, so it is free, deterministic, and always available once classification finishes.

Table 2. Sentiment distribution on the 508-opinion case-study corpus, classified in parallel with GPT-4o-mini.

Source	Positive	Negative	Neutral	Mixed	Unclassifiable	Total
Facebook	3	2	0	1	2	8
TikTok	2	1	6	1	2	12
YouTube	44	275	52	67	50	488
Total	49 (9.6%)	278 (54.7%)	58 (11.4%)	69 (13.6%)	54 (10.6%)	508

3.5 Web Application

The FastAPI backend (`webapp/server.py`) exposes: `POST /api/buscar` to start a job; `GET /api/job/{id}` for polling-based progress, including live per-thread log lines and the pending-login state; `POST /api/job/{id}/confirmar-login` to release a waiting scraper thread; and `GET /api/datasets /api/dataset` to list and load previously generated results. A static dashboard (vanilla JavaScript) consumes this API, letting a user submit a new search, watch extraction and classification progress live, and then explore the resulting corpus with source/sentiment filters, a card and table view, and the storytelling narrative, all without leaving the browser.

4 Results

4.1 Case Study Corpus

The system was evaluated on the public reaction to “*Ecos del Sol*”. A real extraction run consolidated 508 opinions from three concurrently running sources – Facebook (8), TikTok (12), and YouTube (488) – collected under the query “*museo nacional del ecuador*”; the fourth concurrent source described in Section 3 (Reddit, via its official API) extends the same thread-based architecture to a fourth platform and is included in the deployed system, though the corpus analyzed below was captured before that connector was finalized. Classifying the full 508-record corpus in parallel with OpenAI GPT-4o-mini took **49.59 s**, with all three source-threads (`Sentimiento_0/1/2`) starting in the same second, direct evidence that extraction and classification, respectively, ran concurrently rather than sequentially. Table 2 summarizes the resulting sentiment distribution.

The automatically generated storytelling narrative for this run states that the predominant sentiment is negative (54.7%), that YouTube skews far more negative than Facebook or TikTok, and that the highest-engagement comments (ranked by like count) are themselves mostly negative – i.e., the critical view is not only the most frequent one but also the one that resonates most with the audience, a qualitative pattern that a simple aggregate percentage would not surface on its own.

Table 3. Sequential vs. parallel classification benchmark (18 records, 3 producer threads).

Provider	Sequential time	Parallel time	Speedup
OpenAI GPT-4o-mini	21.54 s	7.68 s	2.80 ×
Groq Llama-3.1-8b-instant	6.65 s	26.42 s	0.25 ×

4.2 Execution Time and Parallel Speedup

To isolate the effect of parallelism from network and corpus-size variability, we additionally ran a controlled sequential-vs-parallel benchmark on a fixed 18-record test corpus (6 records per source: Facebook, TikTok, and a third microblogging source), executed once per LLM provider. Table 3 reports the results.

With OpenAI, parallel classification is 2.80× faster than sequential, 93% of the theoretical maximum for three worker threads – strong evidence that the I/O-bound argument from Section 3 holds in practice once the downstream API can absorb concurrent traffic. With Groq’s free tier, however, parallel execution is *four times slower* than sequential (0.25×): three threads issuing requests at the same time exhaust the account’s shared per-minute rate limit almost immediately, and the exponential backoff triggered by repeated HTTP 429 responses adds far more waiting time than the concurrency saves. This is a concrete, measured counter-example to the common assumption that thread-based parallelism is always beneficial for I/O-bound work: when the bottleneck is a *shared, externally rate-limited resource* rather than raw network latency, added concurrency can backfire, and provider-aware throttling or an adaptive number of workers is needed instead of a fixed thread pool.

4.3 Classification Reliability

In the absence of a human-annotated gold-standard label set, we use inter-model agreement between the two supported LLM providers as a proxy for classification reliability, following the evaluation logic used by Kim and Kim [9]. Classifying the same 18-record corpus with both Groq Llama-3.1-8b-instant and OpenAI GPT-4o-mini produces the same label for 15 of 18 records (**83.3% agreement**); the three disagreements were all between adjacent categories (e.g., *negativo* vs. *mixto*), never a positive/negative flip, suggesting that residual disagreement is concentrated on genuinely ambiguous, mixed-stance opinions rather than random classification noise.

5 Conclusions and Recommendations

This paper presented and empirically evaluated a web application that extracts, classifies, and narratively interprets social media opinions across four platforms

using thread-based concurrency at every I/O-bound stage. Four conclusions follow from the design and the results in Section 4:

1. **On natural language processing:** prompting a general-purpose LLM for a strict, five-category JSON sentiment label – rather than using a lexicon-based method or training a dedicated classifier – proved sufficient to capture nuanced, sarcastic, and mixed-stance public commentary in Spanish, at the cost of requiring a reliability proxy (inter-model agreement) in the absence of ground-truth labels.
2. **On concurrency and algorithms:** a shared-lock, shared-event coordination pattern (`GestorLogin`) is enough to let multiple Selenium-driven threads that each require manual login coexist safely with API-driven threads that do not, without serializing the whole pipeline; the same producer–consumer queue pattern used for classification generalizes cleanly across a variable number of sources and workloads of very different sizes (8 to 488 records per source in the case study).
3. **On high-performance/parallel computing:** threads, not processes, are the right tool here because the workload is I/O-bound and benefits from shared memory and GIL release during network waits; but our measured $0.25\times$ speedup with Groq shows that this benefit is conditional on the downstream service’s rate-limiting policy, not a property of the thread pool alone.
4. **On the results themselves:** the case study shows that public sentiment about “Ecos del Sol” is predominantly negative (54.7% of 508 opinions) and that this negativity is concentrated on YouTube and correlates with higher engagement, a pattern that would have been far more costly to establish through manual reading of hundreds of comments across three platforms.

We derive two recommendations for future extensions of this line of work. First, the number of concurrent classification workers should be adaptive to the LLM provider’s actual rate limit (e.g., reducing the thread count or inserting a shared token-bucket limiter when using a free-tier API) instead of fixing it to the number of sources, directly addressing the slowdown measured with Groq. Second, future work should incorporate a small, human-annotated validation sample per case study to complement the inter-model agreement metric with a true accuracy figure, and extend the storytelling module to also report temporal trends once `fecha_publicacion` is populated consistently across all four sources, enabling a before/after narrative around key public events.

References

1. Brown, M.A., Gruen, A., Maldoff, G., Messing, S., Sanderson, Z., Zimmer, M.: Web scraping for research: Legal, ethical, institutional, and scientific considerations. arXiv preprint arXiv:2410.23432 (2024), <https://arxiv.org/abs/2410.23432>
2. Chen, W., Zhou, K., Hu, B., Yang, Y., Xu, Y., Zhuoma, D., Zhu, R., Yang, Y., Pan, J.: Unlocking visitor experiences in cultural heritage sites with SHAP-interpretable AI and social media sentiment analysis. *npj Heritage Science* **13**, 457 (2025). <https://doi.org/10.1038/s40494-025-02014-0>

3. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 4171–4186. Association for Computational Linguistics (2019). <https://doi.org/10.18653/v1/N19-1423>
4. Dixit, A., Byreddy, S., Song, G., Zhu, T.: Data classification with multiprocessing. arXiv preprint arXiv:2312.15152 (2023), <https://arxiv.org/abs/2312.15152>
5. Grattafiori, A., et al.: The Llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024), <https://arxiv.org/abs/2407.21783>
6. Herrera-Poyatos, D., Peláez-González, C., Zuheros, C., Herrera-Poyatos, A., Tejedor, V., Herrera, F., Montes, R.: An overview of model uncertainty and variability in LLM-based sentiment analysis: challenges, mitigation strategies, and the role of explainability. *Frontiers in Artificial Intelligence* **8**, 1609097 (2025). <https://doi.org/10.3389/frai.2025.1609097>
7. Hjelle, S., Mikalef, P., Altwaijry, H., Parida, V.: Organizational decision making and analytics: An experimental study on dashboard visualizations. *Information & Management* **61**(6), 104011 (2024). <https://doi.org/10.1016/j.im.2024.104011>
8. Hutto, C., Gilbert, E.: VADER: A parsimonious rule-based model for sentiment analysis of social media text. In: Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14). vol. 8, pp. 216–225 (2014). <https://doi.org/10.1609/icwsml.v8i1.14550>
9. Kim, K., Kim, S.: Large language models' accuracy in emulating human experts' evaluation of public sentiments about heated tobacco products on social media: Evaluation study. *Journal of Medical Internet Research* **27**, e63631 (2025). <https://doi.org/10.2196/63631>
10. Li, L., Wang, L.: Insights into people's perceptions towards urban public spaces through analysis of social media reviews: A case study of shanghai. *Buildings* **15**(17), 3033 (2025). <https://doi.org/10.3390/buildings15173033>
11. Nielsen, F.Å.: A new ANEW: Evaluation of a word list for sentiment analysis in microblogs. In: Proceedings of the ESWC2011 Workshop on 'Making Sense of Microposts'. pp. 93–98 (2011), <https://arxiv.org/abs/1103.2903>
12. OpenAI: GPT-4o system card. arXiv preprint arXiv:2410.21276 (2024), <https://arxiv.org/abs/2410.21276>
13. Pehlivan, Z., Park, S., Abrahams, A.S., Desblancs-Patel, M., Steel, B.D., Bridgman, A.: Building a media ecosystem observatory from scratch: Infrastructure, methodology, and insights. arXiv preprint arXiv:2506.10942 (2025), <https://arxiv.org/abs/2506.10942>
14. Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A.: A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv preprint arXiv:2402.07927 (2024), <https://arxiv.org/abs/2402.07927>
15. Silberschatz, A., Galvin, P.B., Gagne, G.: Operating System Concepts. Wiley, 10th edn. (2018)
16. Sun, X., Li, X., Zhang, S., Wang, S.: Sentiment analysis through LLM negotiations. arXiv preprint arXiv:2311.01876 (2023), <https://arxiv.org/abs/2311.01876>
17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (2017), <https://arxiv.org/abs/1706.03762>

18. Yang, C., Liu, T.: Social media data in urban design and landscape research: A comprehensive literature review. *Land* **11**(10), 1796 (2022). <https://doi.org/10.3390/land11101796>